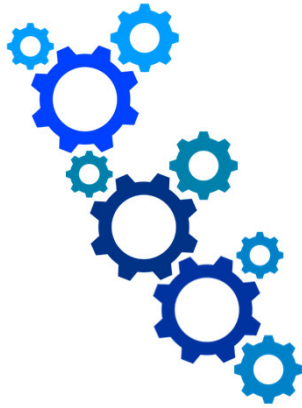


Technologies

Introductory concepts



© M. C. Calzarossa 2026

Enterprise Digital Infrastructure

1

Objectives

Conceptual and implementation aspects of the application layer protocols

Practical experimentations

Competences to be acquired:

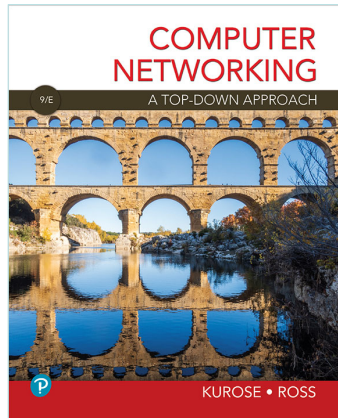
- Solid domain knowledge (i.e., how protocols and technologies work)
- Factors affecting quality
- How to measure the quality attributes

© M. C. Calzarossa 2026

Enterprise Digital Infrastructure

2

Reference book



Chapter 2 Application Layer

Chapter 8 Security in Computer Networks

RFCs

RFC Editor

The Series

[Document Retrieval](#)
[Errata](#)
[FAQ](#)
[xml2rfc FAQ](#)
[Format Change FAQ](#)
[History](#)
[About Us](#)
[Other Information](#)

For Authors

[Publication Process](#)
[Publication Queue](#)
[Style Guide](#)

Mailing Lists

rfc-dist@rfc-editor.org is for RFC publication announcements. rfc-interest@rfc-editor.org is for discussion of the RFC series and related functions.

The RFC series

contains technical and organizational documents about the Internet, including the specifications and policy documents produced by four streams: the Internet Engineering Task Force (IETF), the Internet Research Task Force (IRTF), the Internet Architecture Board (IAB), and Independent Submissions.

Browse the RFC Index

[HTML \(ascending\)](#) • [HTML \(descending\)](#) • [TXT](#) • [XML](#)

Note: These files are large.

Browse RFCs by Status

[Internet Standard](#)

[Draft Standard](#) • [Proposed Standard](#)

[Best Current Practice](#)

[Informational](#) • [Experimental](#) • [Historic](#)

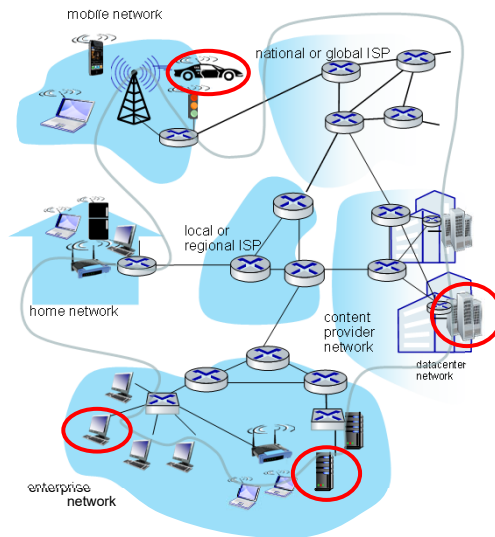
[Uncategorized \(Early RFCs\)](#)

• • • • •

[Official Internet Protocol Standards](#)

[RFC Status Changes](#)

Network applications



Network applications:

distributed applications written in any programming/scripting language (e.g., Java, C/C++/C#, Python, PHP, GO) consisting of *multiple cooperating components* that run on *heterogeneous* end systems and use the *network infrastructure* to communicate

© M. C. Calzarossa 2026

Enterprise Digital Infrastructure

5

Examples of network apps

- email
- web
- instant messaging
- multi-player online games
- real-time video conferencing
- audio/video streaming
- social media
- file sharing
- Internet search
- remote login
- and many many others ...

© M. C. Calzarossa 2026

Enterprise Digital Infrastructure

6

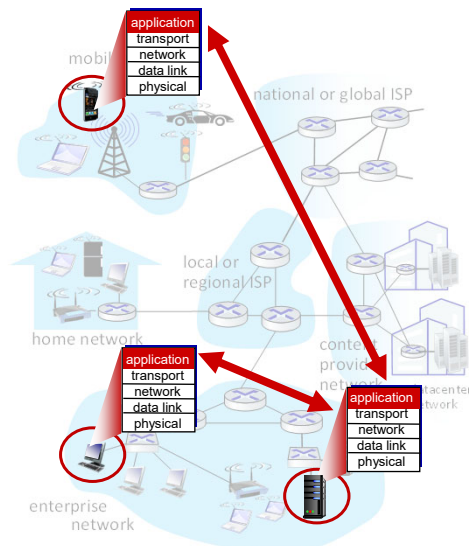
Creating a network application

To create a network app, it is necessary to write programs that:

- run on (different) end systems
- communicate the over network

No need to write software for network-core devices

These devices do not run user applications



© M. C. Calzarossa 2026

Enterprise Digital Infrastructure

7

Architectural models

Two types of components within network applications:

- client components
- server components

An architectural model refers to the organization of these components

The architectural model is a major design choice

Two main architectural models:

1. Client/server model
 - Hybrid model
2. Peer-to-peer (P2P) model

© M. C. Calzarossa 2026

Enterprise Digital Infrastructure

8

Client/server model

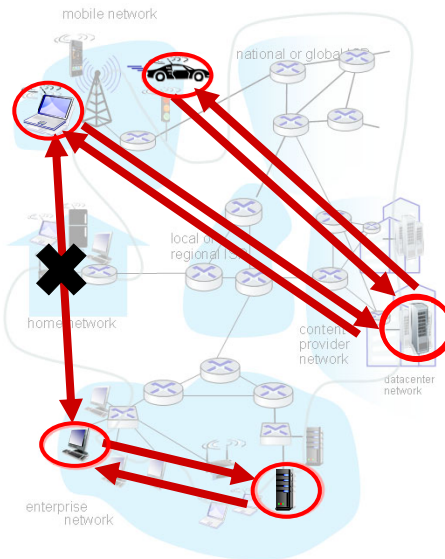
Centralized/hierarchical solution

The server component is the core of the application

The client components request the services provided by the server components

Communications between clients and servers

Examples: Web, email, DNS, ...



© M. C. Calzarossa 2026

Enterprise Digital Infrastructure

9

Role of server & client

Server component runs on a host computer (server)

- Always-on (*availability*)
- Static (*permanent*) IP address(es)
- Cloud infrastructure/data centers to ensure *scalability*

Client components run on user devices

- May be intermittently connected to the Internet
- May have dynamic IP addresses (e.g., assigned by ISPs using DHCP)
- Client components communicate with server component only
- Client components do not communicate directly with other client components

Infrastructure intensive model

© M. C. Calzarossa 2026

Enterprise Digital Infrastructure

10

Peer-to-peer model

Decentralized solution (no hierarchy)

Concept of “peers”, i.e., devices running applications/services

A peer requests services from other peers & provides services in return to other peers

Arbitrary end systems communicate directly

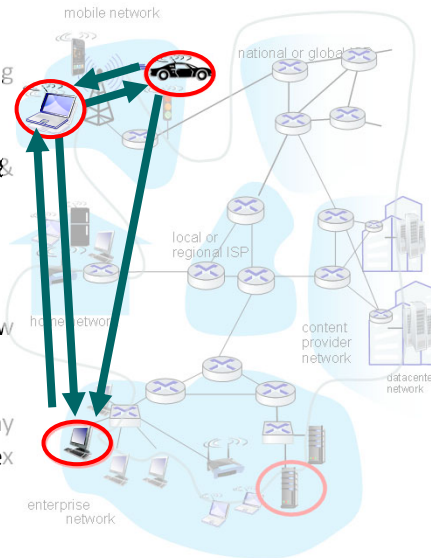
Self scalability (i.e., new peers bring new service capacity and new service demand)

Peers are intermittently connected and may change their IP addresses (complex management)

Examples: file sharing, blockchain, ...

© M. C. Calzarossa 2026

Enterprise Digital Infrastructure



C/S vs P2P

C/S

- Services owned/managed/operated by providers
- Expensive
- Infrastructure intensive model

P2P

- Services operated by the *community* for the *community*
- Cheap
- Concept of sharing (under the assumption of “good” behavior)
- Legal issues (e.g., copyrighted material)
- Security issues (e.g., trust of anonymous peers)
- Technical challenges (e.g., asymmetric bandwidth)
- Lack of incentives to share unused resources

© M. C. Calzarossa 2026

Enterprise Digital Infrastructure

Recap

Network apps consist of multiple *cooperating processes* running on geographically *distributed* systems and organized according to different *architectural models*

- Client/server model
- Peer-to-peer model

Hybrid model

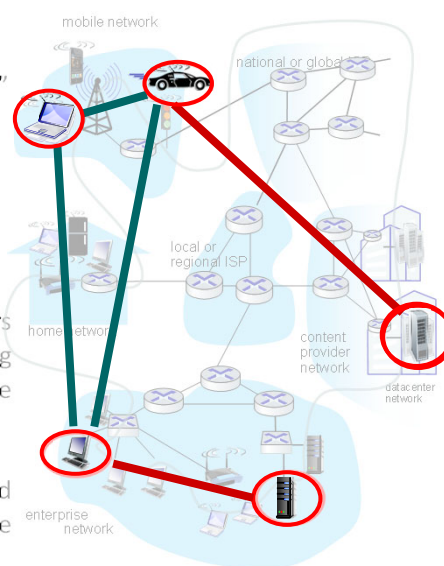
Mixed solution

Applications include (light) “centralized” (i.e., C/S) and “decentralized” components (i.e., P2P)

“Some” control over the service at reduced cost

Zoom platform uses centralized servers in the cloud for authentication, routing and orchestration & P2P for one-on-one conversations

Instant messaging apps use centralized server for client presence detection/location & client-client chats



Implementation of network apps

A *process* is a program running on a host system

Pairs of processes running on the same host communicate using *inter-process communication* mechanisms defined by operating systems (e.g., pipes)

Pairs of processes running on different (heterogeneous) hosts communicate by exchanging *messages*

Client process is the process that initiates the communication (to request the service)

Server process is the process that waits to be contacted (and provides the requested service)

P2P applications include both client and server processes

© M. C. Calzarossa 2026

Enterprise Digital Infrastructure

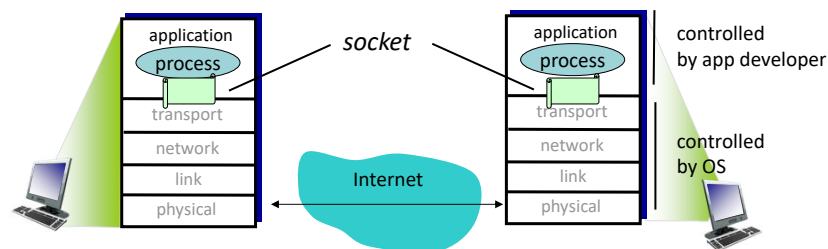
15

Sockets

A process sends/receives messages to/from its *socket* (corresponding to the exit/entry point of the process)

Two sockets involved (one on each side)

Sending process relies on transport infrastructure to deliver messages to the socket at the receiving process



© M. C. Calzarossa 2026

Enterprise Digital Infrastructure

16

Addressing

Each process must be uniquely identified for the message exchange to work

The IP address of the device on which the process is running is necessary for identifying a process, but it is not sufficient

Multiple diverse processes might be running on the same device

The identifier of a process must include both the IP address of the device and the port number associated with the process

On the server side, some port numbers are reserved to specific services (e.g., port 80 for HTTP, port 443 for HTTPS, port 22 for SSH, port 53 for DNS)

On the client side, each client process chooses a (random) port number where it will expect to receive responses from the server

© M. C. Calzarossa 2026

Enterprise Digital Infrastructure

Application requirements on transport service

Developers need to choose the most suitable transport protocol for their applications

The choice depends on QoS requirements of the applications with respect to data transfer

QoS requirements (wish list) are defined in terms of four dimensions:

- **Data integrity:** the reliability in the transfer between sending and receiving processes (i.e., tolerance to data losses)
- **Throughput:** the rate at which sending process can deliver data to receiving process (i.e., bandwidth requirements)
- **Time/delay:** the end-to-end delay between sending and receiving processes (i.e., timing requirements)
- **Security:** the privacy/confidentiality of data being transmitted (i.e., requirements on data in transit against security threats, such as eavesdropping, tampering and forgery)

© M. C. Calzarossa 2026

Enterprise Digital Infrastructure

Transport service requirements: examples

application	data loss	throughput	time sensitive?
file transfer	no loss	elastic	no
e-mail	no loss	elastic	no
web browsing	no loss	elastic	no
real-time audio/video	loss-tolerant	audio: 5K-1Mbps video: 10K-5Mbps	yes, 10's msec
streaming	loss-tolerant	same as above	yes, few secs
interactive games	loss-tolerant	Kbps+	yes, 10's msec
text messaging	no loss	elastic	yes and no

© M. C. Calzarossa 2026

Enterprise Digital Infrastructure

19

Transport service protocols

application	application layer protocol	transport layer protocol
file transfer	FTP	TCP
e-mail	SMTP	TCP
web browsing	HTTP	TCP (or UDP)
real-time audio/video	proprietary	UDP
streaming	HTTP, DASH	TCP
interactive games	proprietary	UDP or TCP
text messaging	proprietary	TCP or UDP

© M. C. Calzarossa 2026

Enterprise Digital Infrastructure

20

Application layer protocols

Protocols specify

- *types* of messages (i.e., request, response) exchanged between processes
- *syntax* (i.e., fields, delimiters)
- *semantics* (i.e., meaning)
- *rules* and *actions* (i.e., when and how processes send and respond to messages)

Two categories of protocols:

- *Open protocols* whose characteristics/behavior are fully specified within public documents (e.g., RFCs) to ensure interoperability
- *Proprietary protocols* whose specifications are not made publicly available to “protect” business (e.g., competitiveness)